

SUMMARY: INTRODUCTION TO RAG & TEXT REPRESENTATION

Objective:

By the end of this session, students will be able to:

1. Explain what Retrieval-Augmented Generation (RAG) is and why it matters in modern AI systems.
2. Describe how RAG combines searching for relevant content with natural language generation.
3. Differentiate between parametric (model-internal) and non-parametric (external knowledge base) memory.
4. Understand the limitations of tokens and traditional Bag of Words/TF-IDF methods for finding relevant information.
5. Define vector embeddings and explain how they help measure similarity between queries and documents.
6. Demonstrate how to retrieve relevant documents using semantic search and sentence embeddings.
7. List and describe common strategies for selecting and summarizing relevant information in RAG pipelines.

Introduction to RAG

You're chatting with an AI. You ask, "What are the health benefits of yoga?" The AI doesn't just guess from what it remembers—it searches for the most relevant, up-to-date information, then explains it as if it's a human expert.

How does it do that? That's the secret of RAG.

What is Retrieval-Augmented Generation (RAG)?

Most people believe that Large Language Models (LLMs) like ChatGPT or Gemini "know everything."
But here's the truth:

LLMs are like super-smart students who read millions of books—but even they can't remember *everything*, especially recent news or your company's data.

This is where RAG comes in.

RAG is a method where the AI doesn't just rely on its memory. Instead, it 'retrieves' the most relevant information from a large set of documents or a database—then uses that information to generate a clear, accurate answer.

The Core Idea

RAG = Search + Generation

- Search: The AI first finds the most useful pieces of information from a massive library—maybe a database, a knowledge base, or even the internet.
- Generation: Next, the AI weaves those pieces together into a fluent, human-like answer.

Why Does RAG Matter?

- **More up-to-date:** RAG lets AI answer questions about *yesterday's news* or *your company files*—things it didn't "know" during training.
- **More accurate:** It reduces "hallucinations," where AI makes things up.
- **More personal:** It can answer using your company's private data, securely.

A Quick Story

An HR manager at a big firm needs the exact vacation policy for contract workers. She asks the company's AI assistant, which uses RAG.

- The assistant searches the latest HR documents.
- It finds the 2024 contract staff policy.
- It replies: "According to the 2024 policy, contract workers are eligible for 12 days of paid leave per year."

Without RAG, the assistant might have given an outdated or generic answer.

How AI Responds *Without* RAG

When an AI does **not** use RAG, it does **not** actually "remember" facts or training data like a human memory. Instead, it **predicts the next word** based on patterns it learned during training.

For example:

You ask:

“What is the leave policy for contract employees in our company?”

AI Response (WITHOUT RAG):

“Contract employees may have different leave entitlements compared to full-time staff. Generally, leave policies can vary depending on the organization. Please check with your HR department for details.”

What really happened here?

The AI is not pulling from your company’s current documents or policies.

Instead, it is simply **predicting a likely, generic answer** by looking at how similar questions were answered in its past training examples.

Vector Embeddings

Introduction to Embeddings

How AI “Understands” and Connects Meaning

Now our task is to find similar knowledge to the query which is entered in an LLM, so how do we find these similar or relevant pieces of information?

We already know that words are represented as tokens, so we can use tokens to find similar looking information. But there is a drawback of tokens.

Why Tokens Fall Short

- When AI reads, it breaks text into tokens—like “cat,” “dog,” “saxophone.”
- But these tokens are just arbitrary numbers (like “1121,” “9041,” “223”).
- The number for “cat” might be 1121, and for “dog” it could be 9041. The numbers themselves give no clue that “cat” and “dog” are more similar than “cat” and “saxophone.”
- That’s because tokens are only labels for “word pieces,” not indicators of meaning.

Why Tokens Aren’t Enough

When you type a sentence to an AI, the first thing it does is break it into tokens.

For example, the phrase “**A dog chased a cat**” might get broken into tokens like [102, 982, 4001, 102, 512].

But here’s the problem:

- The number 982 (for “dog”) is just an ID, not a *meaningful* number.
- The number 512 (for “cat”) might be far away in the ID sequence, even though in real life, “dog” and “cat” are closely related concepts.

This is like assigning every book in the library a random number. “Cats” could be #982, “Dogs” could be #4001, “Cars” could be #12. If you try to look up similar books by number, you’d never find related topics together.

Tokens help break down language, but they cannot capture *context* or *similarity*.

A Need for a Better Representation

So, what do we do if we want an AI to recognize that “cat” and “dog” are similar, but “cat” and “car” are not?

We need a smarter scheme—a way of turning words, phrases, or even whole documents into numbers that actually capture their *meaning* and *context*.

Think about arranging books on a single line (a 1-dimensional shelf), not by title, but by topic similarity. Let’s assign each animal a spot:

- **cat** = 2.1
- **dog** = 2.3
- **tiger** = 2.7
- **car** = 9.5

Now, “cat” and “dog” are neighbors. “Car” is far away.

This is already better—but with just one dimension, there’s only so much nuance we can capture. What about all the shades of meaning between “cat” and “tiger,” or “cat” and “pet,” or “cat” and “instrument” (as in “catgut”)?

This is called **semantic similarity**.

Stepping Into the World of Vectors

Let’s take it further. Imagine not a line, but a map—where every word is a dot placed somewhere in space. The closer the dots, the more similar the words.

This is where *vector embeddings* come in.

Every word, sentence, or document is turned into a vector—a list of numbers, like coordinates. In 2D,

“cat” might be at (1, 1), “dog” at (1.1, 1), and “car” at (7,8).

Now, AI can actually measure which words are “close” in meaning, just by looking at the distance between these dots.

If someone asks your AI assistant about “cat,” it can instantly find not just “cat,” but also “dog,” “kitten,” or “pet”—because these live close together on this map.

If someone asks about “car,” it won’t get confused, because “car” is off in another part of the map, far from “cat.” [Refer to figure below]

Beyond Our Imagination: High-Dimensional Embeddings

Real AI doesn’t just use 2D or 3D maps. It uses spaces with *hundreds or thousands of dimensions*—so it can capture every subtlety, from grammar to intent, topic to tone.

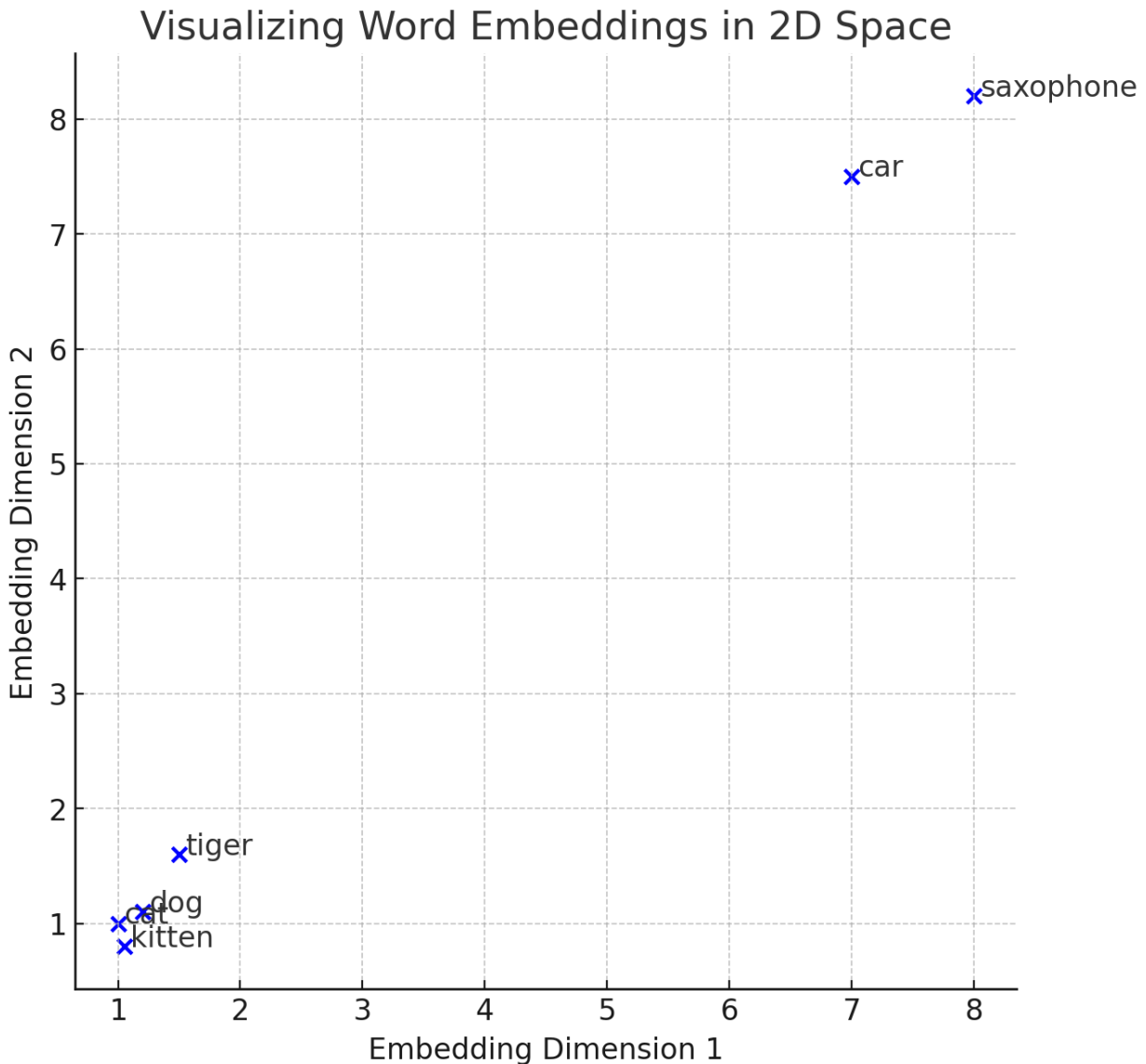
You can’t visualize this, but the core idea remains:

Similar meaning → similar vectors → close together in space.

This is the secret behind how AI retrieves the most relevant content for your query—even if you’ve never asked that exact question before.

Visualizing Embeddings

Here’s a diagram showing how “cat” and “dog” cluster together, while unrelated words like “car” and “saxophone” are far apart.



Embeddings Explained

Researchers wanted machines to not just see words, but to **feel their meaning**—to spot the invisible threads connecting language, just as we do.

They started with a simple but clever idea:

If you read enough books, newspapers, and all of Wikipedia, you start noticing patterns. When you see the sentence,

“A cat chased the ...”

you instinctively know “dog,” “mouse,” or “kitten” could fit at the end. But “car” or “saxophone” probably would not.

So, researchers built models that did just this:

They trained neural networks to *predict the next word* in millions of real sentences from the world's largest text libraries.

The Big Surprise:

These models learned to *connect words that often appear in similar contexts*. They realized that, inside the neural network, some layers had quietly learned a kind of “map” for each word—a vector, a list of numbers, capturing its meaning and relationships.

For example, for the word “cat”, deep inside the network you might find something like:

[0.25, -1.7, 0.84, 2.1]

While for “dog” you might see:

[0.30, -1.5, 0.90, 2.0]

And for “car”:

[5.2, 2.1, -3.4, 0.7]

See how “cat” and “dog” are almost twins, while “car” is off in a different neighborhood entirely? This *list of numbers*—this vector—is the **embedding** for the word.

But what’s magical:

Even if “cat” and “kitten” never appeared together in a single sentence, if they were used in similar kinds of sentences—

“She adopted a kitten from the shelter.”

“She adopted a cat from the shelter.”

—then the model will pull their vectors close together in the embedding space.

Here’s a vivid analogy:

It’s like plotting every word in the world on a map, not by where they’re written, but by *how they’re used*. Two words might live in completely different parts of town, but if people talk about them in the same way, they end up as neighbors.

This is why, when you ask an AI for “kitten care tips,” it knows to also bring in advice written for “cats”—because their embeddings are close, even if “kitten” and “cat” never shared a page.

And what did researchers call this early approach?

One of the most famous models to do this was **Word2Vec**, built by a team at Google in 2013. Word2Vec trained on all of Wikipedia, predicting words from context, and then used the “hidden layers” as word vectors—creating the first widely-used word embeddings.

Let’s peek at what an embedding looks like:

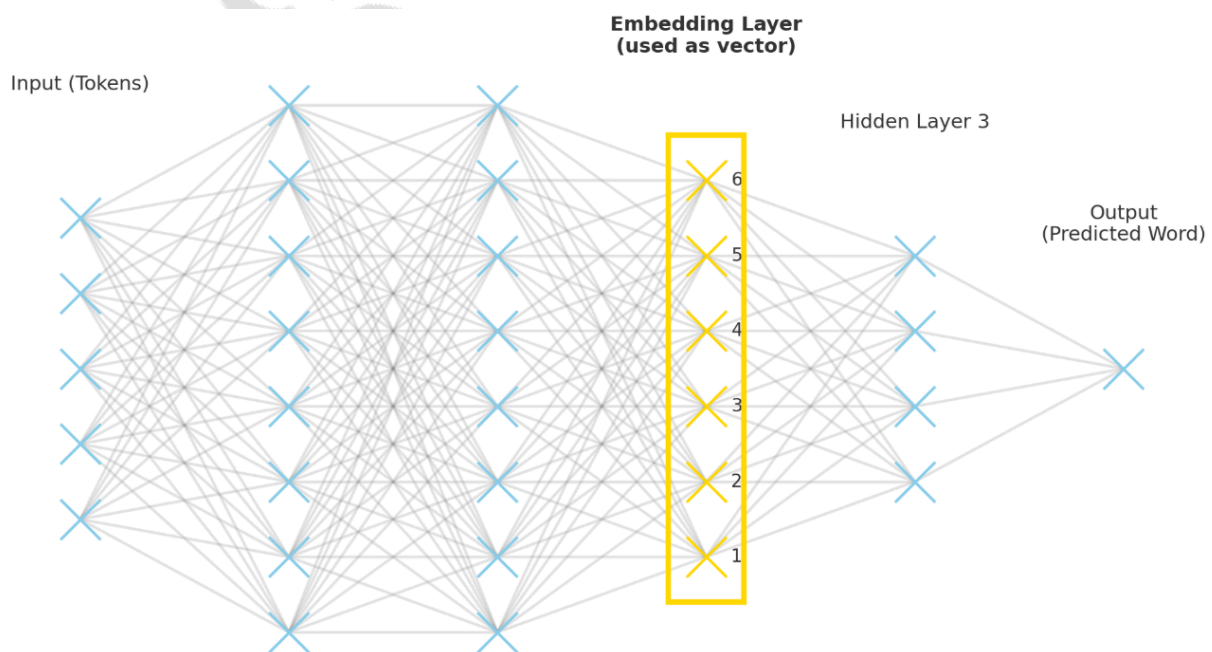
- “cat” = [0.3, -1.2, 0.4, 1.7, ...]

- “dog” = [0.4, -1.1, 0.5, 1.6, ...]
- “saxophone” = [6.8, 5.2, -0.9, -2.1, ...]

Just a bunch of numbers? Yes.

But with these, an AI can measure closeness, find similarity, and connect words in a way tokens never could.

So, embeddings are the invisible map that lets AI understand the meaning behind words—even when words have never shared the same sentence, as long as their context is similar, they’ll be mapped close together in the AI’s world.



The Story Behind Famous Embeddings

Let’s step through the evolution of word embeddings. Each of these models represents a leap in how AI “understands” meaning—not just as a technical advance, but as a story of curiosity and innovation.

1. Word2Vec (Google, 2013): Predicting Words from Their Neighbors

The Story:

A team at Google, led by Tomas Mikolov, asked:

What if we train a neural network to predict a missing word, given the words around it?

They fed the model with billions of sentences from Wikipedia and news. The network's only job was to guess words like a "fill in the blank" puzzle:

"The cat sat on the ____."

By training on this simple task, they found that **one of the hidden layers—just before the output—encoded each word as a vector**. These vectors grouped together words with similar meanings.

Suddenly, you could do things like:

King - Man + Woman $\approx$ Queen

It was the first time AI found relationships between words *numerically*.

2. GloVe (Global Vectors, Stanford, 2014): Counting Word Pairs

The Story:

The Stanford NLP team took a new angle:

What if we just look at how often words appear together in the same context, across all of Wikipedia?

Instead of predicting a missing word, **GloVe** built a giant matrix counting how often every pair of words showed up together ("cat" and "fur," "doctor" and "hospital").

They used clever math (matrix factorization) to compress this information into compact vectors.

The result: **Word vectors where distance reflects how strongly words are associated in real life.**

3. FastText (Facebook AI, 2016): Understanding Subwords

The Story:

Facebook engineers noticed a problem:

What if a word isn't in our dictionary? Like "embeddify" or a new name?

FastText solved this by breaking words into chunks—like "cat," "cats," "catlike."

Instead of just one vector per word, FastText averaged vectors for all the *pieces* (subwords). This meant it could generate embeddings for brand-new words, as long as their pieces were known.

4. ELMo (Allen Institute, 2018): Context Matters

The Story:

The Allen Institute team realized:

Words change meaning depending on the sentence.

Compare "bank" in

"He sat by the river bank."

vs.

"She works at the bank."

ELMo used deep neural networks to generate a new embedding for every word in every

sentence—fully *contextual*.

So, “bank” in both sentences would get different vectors, based on its neighbors.

5. BERT (Google, 2018): Deep Context, Deep Connections

The Story:

BERT (Bidirectional Encoder Representations from Transformers) marked a revolution.

Instead of just reading left-to-right or right-to-left, BERT read both ways at once—capturing full sentence meaning.

It was trained to “fill in the blank” in the *middle* of sentences and to decide if two sentences followed each other logically.

“The [MASK] sat on the mat.”

BERT would learn the “cat” goes here.

BERT’s embeddings are powerful, context-aware, and form the backbone of modern AI systems.

How Are Embeddings Created?

- **Massive text data** (Wikipedia, news, books) is fed into neural networks.
- The model is trained on a language task—predicting missing words, counting co-occurrences, or understanding context.
- The “embedding layer” (one of the last hidden layers) is saved as the vector representation for each word, phrase, or even whole sentences.
- These vectors become the “map” that lets AI compare, relate, and retrieve meaning.

```
from gensim.models import Word2Vec
from sklearn.metrics.pairwise import cosine_similarity
import numpy as np

# Sample "documents"
documents = [
    "Cats are playful pets.",
    "Dogs are loyal animals.",
    "Electric cars are the future of transportation."
```

```
]

# Preprocess: tokenize documents into words
tokenized_docs = [doc.lower().replace('.', ' ').split() for doc in
documents]

# Train a Word2Vec model on the documents
model = Word2Vec(sentences=tokenized_docs, vector_size=50, window=2,
min_count=1, workers=1, sg=1)

# Function to get the average embedding for a document
def get_avg_embedding(model, doc_tokens):
    embeddings = [model.wv[word] for word in doc_tokens if word in
model.wv]
    return np.mean(embeddings, axis=0) if embeddings else
np.zeros(model.vector_size)

# Get embeddings for each document
doc_embeddings = np.array([get_avg_embedding(model, doc) for doc in
tokenized_docs])

# Let's use the query "cat" and get its embedding
query_tokens = ["cat"]
query_embedding = get_avg_embedding(model, query_tokens)

# Compute cosine similarity between query and each document
sims = cosine_similarity([query_embedding], doc_embeddings)[0]

# Print the most relevant document
most_similar_idx = np.argmax(sims)
print("Query:", "cat")
print("Most relevant document:", documents[most_similar_idx])
```

Types of Embeddings in Natural Language Processing

1. Word Embeddings

Word embeddings assign each word in a vocabulary a single vector of real numbers. These vectors are trained so that words used in similar contexts have similar vectors.

- **How they're made:** Models like Word2Vec and GloVe read massive corpora and learn from word co-occurrence patterns.
- **Properties:** Every occurrence of a word gets the *same* vector, regardless of context.
- **Example:**
 - “cat” and “dog” have similar vectors; “car” is far from both.

2. Subword Embeddings

Subword embeddings break words into smaller units, such as prefixes, roots, or suffixes. This helps the model handle new or rare words.

- **How they're made:** Models like FastText learn vectors for both whole words and subwords, then combine them for out-of-vocabulary terms.
- **Properties:** Can create vectors for new words by averaging subword pieces.
- **Example:**
 - “unhappiness” → “un”, “happy”, “ness” combined to form a new embedding.

3. Character Embeddings

Character embeddings represent words as sequences of character-level vectors. This approach can capture misspellings or morphologically rich languages.

- **Use:** Often used in models that need to deal with a lot of misspelled or morphologically complex words.

4. Sentence Embeddings

Sentence embeddings encode an entire sentence into a single vector.

One simple way to create a sentence embedding is to **take the average of the embeddings of all the words in the sentence**.

How it works (Averaging Word Embeddings):

Suppose we have the following sentence:

“The dog chased the cat.”

Let's say, for illustration, our word embeddings (with 3 dimensions) look like this:

Word	Embedding
the	[0.2, 0.1, 0.0]
dog	[1.0, 0.8, 0.9]
chased	[0.7, 0.4, 0.6]
cat	[0.9, 1.1, 0.8]

To get the sentence embedding, we:

1. Take the embeddings of each word in the sentence.
2. Add them up, element-wise.
3. Divide by the number of words.

Calculation:

Add the vectors:

- [0.2, 0.1, 0.0] (the)
- [1.0, 0.8, 0.9] (dog)
- [0.7, 0.4, 0.6] (chased)
- [0.2, 0.1, 0.0] (the) *(again, if "the" repeats)*
- [0.9, 1.1, 0.8] (cat)

Sum:

[0.2 + 1.0 + 0.7 + 0.2 + 0.9,
0.1 + 0.8 + 0.4 + 0.1 + 1.1,

$$0.0 + 0.9 + 0.6 + 0.0 + 0.8] \\ = [3.0, 2.5, 2.3]$$

Divide by number of words (5):

- $[3.0/5, 2.5/5, 2.3/5]$
- $[0.6, 0.5, 0.46]$

So, the sentence embedding for “The dog chased the cat.” is:
 $[0.6, 0.5, 0.46]$

Why does this work?

Averaging word embeddings pulls together the “mean” meaning of all the words in the sentence, so sentences with similar words and structure will have close vectors.

So,

- “The dog chased the cat.”
- “A dog ran after a cat.”
would have embeddings with similar values, since their word vectors are close.

More advanced models now use neural networks to get context-aware sentence embeddings, but averaging word vectors is a classic and intuitive starting point!

Averaging word embeddings is a simple and effective way to represent a sentence, but it has some big limitations:

- **No sense of order:** “Dog bites man” and “Man bites dog” get the same embedding if you just average.
- **No context:** The word “bank” in “river bank” and “bank account” is represented by the same vector—even though the meanings are totally different.
- **Ignores grammar and relationships:** Averaging can’t capture how words interact in a sentence.

How do advanced models improve on this?

Instead of just averaging, **neural networks** process the entire sentence as a sequence. This allows them to:

- Capture **word order**
- Recognize **context** and how the meaning of a word changes depending on its neighbors

- Understand **relationships** (who is doing what to whom)

Famous Examples of Context-Aware Sentence Embeddings

1. ELMo (Embeddings from Language Models)

- ELMo generates embeddings for each word *based on the whole sentence*, so “bank” in “river bank” is different from “bank” in “bank loan.”
- To get a sentence embedding, you can average the ELMo word vectors (already context-aware), or use the representation of the whole sentence.

2. BERT (Bidirectional Encoder Representations from Transformers)

- BERT reads the sentence both left-to-right and right-to-left at the same time.
- It gives a unique vector for each word, in context.
- For sentence embeddings, people often use the embedding of the special [CLS] token or average all word embeddings from the last layer.

Example:

- **“The bank was flooded after the heavy rain.”**
The embedding for “bank” will be close to “river” and “shore.”
- **“He deposited money in the bank.”**
The embedding for “bank” here will be close to “money,” “finance,” “account.”

3. Sentence Transformers (Sentence-BERT, SBERT)

- Sentence-BERT is designed to produce high-quality sentence embeddings for tasks like semantic search, duplicate question detection, and clustering.
- It modifies BERT so that an entire sentence is encoded as a single vector, specifically optimized so that similar sentences are close together.

Example:

- **“How do I reset my password?”**
- **“What should I do if I forget my login password?”**
Both get sentence embeddings that are nearly identical.

Why does this matter?

- With context-aware models, AI systems can distinguish not only meaning but intent, tone, and relationships.
- This is critical for applications like document search, Q&A systems, chatbots, translation, and summarization—where “close enough” isn’t good enough!

Summary

- **Averaging word embeddings:** Fast, simple, but ignores context and order.
- **Neural models (ELMo, BERT, SBERT, etc.):** Capture the full meaning, relationships, and context of each word and the sentence as a whole—enabling AI to truly “understand” language in a way that’s much closer to how humans do.

5. Contextual Embeddings

Contextual embeddings give different vectors to the same word depending on its sentence.

- **How they’re made:** Deep models like ELMo, BERT, and GPT produce embeddings that reflect the context.
- **Properties:** “Bank” in “river bank” and “bank account” get different embeddings.
- **Use:** Modern NLP tasks, QA, and anything needing nuanced understanding.

6. Document Embeddings

Document embeddings represent an entire paragraph, article, or document as a single vector.

- **How they’re made:** Models aggregate sentence or word embeddings (or use models trained specifically for documents, like Doc2Vec).
- **Use:** Document retrieval, clustering, topic modeling.

How was text represented before Neural Networks/Embeddings

Long before neural networks and embeddings, search engines like Google were already clustering similar documents and putting related links together on results pages. But how did they do it?

The “Bag of Words” Era

Imagine you’re a librarian in the 1990s. You want to help people find articles about “cat care.” Instead of understanding meaning, you just **count how many times each word appears** in a document.

Bag of Words (BOW):

- Each document is represented as a big list (or “bag”) of its words, with the count of each.
- Word order doesn’t matter; only frequency does.

Example:

Suppose you have these two documents:

- Doc 1: "Cats are lovely pets."
- Doc 2: "Dogs and cats make great pets."

The BOW vectors for the vocabulary {cats, dogs, lovely, are, and, make, great, pets}:

Word	Doc 1	Doc 2
cats	1	1
dogs	0	1
lovely	1	0
are	1	0
and	0	1
make	0	1
great	0	1
pets	1	1

So, each document is a vector of counts.

When a user searches for “cat pets”, the search system represents this query as a similar vector and finds documents with the most matching words.

Why Bag of Words Falls Short

- It doesn't understand meaning or synonyms (“kitty” vs “cat”).
- Very common words (like “the”, “and”, “are”) can dominate the representation, even though they don't tell you much about the document's topic.

TF-IDF: Giving Weight to Important Words

To fix these problems, **TF-IDF (Term Frequency – Inverse Document Frequency)** was invented.

TF-IDF scores words higher if they are *important* to a particular document, and lower if they are common across all documents.

How it works:

- **Term Frequency (TF):** How often does a word appear in this document?
- **Inverse Document Frequency (IDF):** Is this word rare or common across all documents?

Formula:

$$\text{TF-IDF} = \text{TF} * \text{IDF}$$

Where:

- $\text{TF} = (\text{Number of times word appears in document}) / (\text{Total words in document})$
- $\text{IDF} = \log_e(\text{Total number of documents} / \text{Number of documents containing the word})$

So, a word like “cat” will get a higher score in a document all about cats, but a common word like “the” will get a low score everywhere.

Example:

Suppose we have three documents:

- Doc 1: “Cats are great pets.”
- Doc 2: “Dogs are great pets.”
- Doc 3: “Cats and dogs can be friends.”

Let’s see how TF-IDF helps a search for “cats friends”.

- The word “cats” might have a high TF-IDF in Doc 1 and Doc 3, but low in Doc 2.
- The word “friends” might have high TF-IDF only in Doc 3.
- The word “are” has low TF-IDF everywhere (since it appears in all documents).

So, for the query “cats friends”, Doc 3 would score highest and appear first—because it uses both rare, relevant words.

Retrieval-Augmented Generation (RAG): Detailed Explanation

Step 1: Pulling Relevant Documents Using Sentence Embeddings - RETRIEVER

First, we need to retrieve relevant information from a collection of documents using semantic similarity. Let’s use BERT-based sentence embeddings (e.g., via [sentence-transformers](#)) to demonstrate:

```
from sentence_transformers import SentenceTransformer, util

# Example documents
docs = [
    "Cats are playful and independent pets.",
    "Electric cars are eco-friendly vehicles.",
    "Dogs are loyal and friendly animals.",
    "Kitten care includes feeding, grooming, and vaccinations."
]

# Initialize the model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Encode all documents
doc_embeddings = model.encode(docs, convert_to_tensor=True)

# Query
query = "How do I take care of a kitten?"
```

```
# Encode query
query_embedding = model.encode(query, convert_to_tensor=True)

# Find most similar docs
cos_scores = util.cos_sim(query_embedding, doc_embeddings)[0]
top_results = cos_scores.argsort(descending=True)[:2]

for idx in top_results:
    print(f"Score: {cos_scores[idx]:.4f} Document: {docs[idx]}")
```

What happens here:

- We encode the query and all docs into embeddings.
- We use cosine similarity to find which documents are *truly related* to the question—even if exact words don't match.
- The most relevant docs (e.g., about kitten care) are selected.

Step 2: Using Retrieved Docs in a Generation Model - GENERATOR

After retrieving, we pass the relevant docs (along with the original question) to a **generator**—an LLM like GPT, Gemini, or Llama-2.

How this looks:

- The LLM gets a prompt that combines the user's question and the retrieved context.

Prompt example for the generator:

```
Context:
"Kitten care includes feeding, grooming, and vaccinations."
"Cats are playful and independent pets."

Question: How do I take care of a kitten?

Answer:
```

Now, the LLM reads the *relevant information* before generating an answer—reducing hallucination and grounding its response in facts.

So basically we're giving the content of relevant document as prompt to the LLM.

Should We Give the Entire Content or Just a Summary?

When building RAG systems, we don't always give the LLM the entire text of every relevant document. There are practical and strategic reasons for this:

1. Context Window Limit

LLMs have a maximum number of tokens ("context window") they can process at once. For most models, this ranges from a few thousand to maybe 100,000 tokens (for the biggest models). If documents are long, we can't fit everything into one prompt.

2. Relevance and Focus

Not every part of a document is useful for every question. If we include too much, the answer can get diluted or confused.

Common Strategies in RAG Systems:

- **Chunking:**
Break large documents into smaller "chunks" (paragraphs, sections, etc.), and retrieve only the most relevant chunks based on the query.
- **Summarization:**
Sometimes, retrieved content is summarized—either by a model or an algorithm—before being given to the LLM. This helps compress large info into a concise, focused context.
- **Top-K Selection:**
Only the top K (for example, top 3 or top 5) most relevant chunks are included in the prompt to the LLM.
- **Reranking:**
Retrieved chunks can be re-scored and reranked, to ensure only the highest quality/relevance is passed on.
- **Hybrid approaches:**
Some systems summarize the retrieved content further if it still doesn't fit in the context window.

In summary:

Most RAG systems do *not* provide the full text of entire documents. Instead, they use smart strategies to

select and condense only the most relevant, useful parts—making sure the LLM receives just enough information to answer accurately, without exceeding the model’s limits or overwhelming it with noise.

Summary:

Retriever = “Find the right pages.”

Generator = “Read and answer using those pages.”

How External Knowledge Bases look like for LLMs

A typical RAG system allows the LLM to “look outside itself” using an **external knowledge base**. This base might be:

- Company documents
- PDFs, manuals, or FAQs
- Internal wikis
- Recent news
- Custom datasets

These sources are indexed using embeddings or other retrieval methods, so the system can always pull in up-to-date or domain-specific info. [to be discussed in future sessions]

In summary:

RAG systems work by using a retriever to fetch relevant, trustworthy context for each user question, then a generator LLM to create an answer grounded in that context.

This makes AI systems far more reliable, updatable, and useful in real-world applications.